

National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

S8 Streams

Problems:

Getting started

- Describe the streams A and B produced by the following definitions. Assume that `integers` is the stream of positive integers (starting from 1):

```
def scale_stream(c, stream):
    return stream_map(lambda x: c * x, stream)
```

```
A = pair(1, lambda: scale_stream(2, A))
```

```
def mul_streams(a, b):
    return pair(head(a) * head(b),
                lambda: mul_streams(stream_tail(a), stream_tail(b)))
```

```
B = pair(1, lambda: mul_streams(B, stream_tail(integers)))
```

Using streams to represent power series

- The following power series

$$e^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3 \cdot 2} + \frac{x^4}{4 \cdot 3 \cdot 2} + \dots$$

$$\cos x = 1 - \frac{x^2}{2} + \frac{x^4}{4 \cdot 3 \cdot 2} - \dots$$

$$\sin x = x - \frac{x^3}{3 \cdot 2} + \frac{x^5}{5 \cdot 4 \cdot 3 \cdot 2} - \dots$$

can be represented as streams of infinitely many terms. That is, the power series

$$a_0 + a_1x + a_2x^2 + a_3x^3 + \dots$$

will be represented as the infinite stream whose elements are $a_0, a_1, a_2, a_3, \dots$ ¹ Note that x does not appear in the representation at all: a series is just its stream of coefficients. A value for x only comes in when we *evaluate* the series, as shown next.

¹In this representation, all streams are infinite: a finite polynomial will be represented as a stream with an infinite number of trailing zeroes.

Why would we want such a method? Well, let's separate the idea of a series representation from the idea of evaluating a function. For example, suppose we let $f(x) = \sin x$. We can separate the idea of evaluating f , e.g., $f(0) = 0, f(.1) = 0.0998334$, from the means we use to compute the value of f . This is where the series representation is used, as a way of storing information sufficient to determine values of the function. In particular, by substituting a value for x into the series, and computing more and more terms in the sum, we get better and better estimates of the value of the function for that argument. This is shown in the table, where $\sin \frac{1}{10}$ is considered.

Coefficient	x^n	term	sum	value
0	1	0	0	0
1	$\frac{1}{10}$	$\frac{1}{10}$	$\frac{1}{10}$	.1
0	$\frac{1}{100}$	0	$\frac{1}{10}$	.1
$-\frac{1}{6}$	$\frac{1}{1000}$	$-\frac{1}{6000}$	$\frac{599}{6000}$	.099833333333
0	$\frac{1}{10000}$	0	$\frac{599}{6000}$	.099833333333
$\frac{1}{120}$	$\frac{1}{100000}$	$\frac{1}{12000000}$	$\frac{1198001}{12000000}$	.09983341666

The first column shows the terms from the series representation for sine. This is the infinite series with which we will be dealing. The second column shows values for the associated powers of $\frac{1}{10}$. The third column is the product of the first two, and represents the next term in the series evaluation. The fourth column represents the sum of the terms to that point, and the last column is the decimal approximation to the sum.

With this representation of functions as streams of coefficients, series operations such as addition and scaling (multiplying by a constant) are identical to the basic stream operations. We provide series operations, though, in order to implement a complete power series data abstraction:

```
def add_streams(s1, s2):
    return (s2
            if is_none(s1)
            else s1
            if is_none(s2)
            else pair(head(s1) + head(s2),
                      lambda: add_streams(stream_tail(s1),
                                           stream_tail(s2))))

def scale_stream(c, stream):
    return stream_map(lambda x: c * x, stream)

add_series = add_streams

scale_series = scale_stream

def negate_series(s):
    return scale_series(-1, s)

def subtract_series(s1, s2):
    return add_series(s1, negate_series(s2))
```

We also provide two ways to construct series. The function `coeffs_to_series` takes a linked list of initial coefficients and pads it with zeroes to produce a power series. For example,

```
coeffs_to_series(llist(1, 3, 4))
```

produces the power series $1 + 3x + 4x^2 + 0x^3 + 0x^4 + \dots$.

```
def coeffs_to_series(llist_of_coeffs):
    zeros = pair(0, lambda: zeros)

    def iter(lst):
        return (zeros
                if is_none(lst)
                else pair(head(lst),
                        lambda: iter(tail(lst))))
    return iter(llist_of_coeffs)
```

The function `fun_to_series` takes as argument a function p of one numeric argument and returns the stream $p(0), p(1), p(2), p(3), \dots$, which represents the power series

$$p(0) + p(1)x + p(2)x^2 + p(3)x^3 + \dots$$

The definition requires the stream `non_neg_integers` to be the stream of non-negative integers: $0, 1, 2, 3, \dots$.

```
def fun_to_series(fun):
    return stream_map(fun, non_neg_integers)
```

To get some initial practice with streams, write definitions for each of the following:

- `alt_ones`: the stream $1, -1, 1, -1, \dots$ in as many ways as you can think of.
- `zeros`: the infinite stream of 0's. Do this using `alt_ones` in as many ways as you can think of.

Now, show how to define the series:

$$\begin{aligned} S_1 &= 1 + x + x^2 + x^3 + \dots \\ S_2 &= 1 + 2x + 3x^2 + 4x^3 + \dots \end{aligned}$$

Finally, write a function `eval_series(s, x, n)` that evaluates the series s at x , using the first n terms $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$. Test it on S_1 and S_2 with $x = 0.5$, and compare the results with $\frac{1}{1-x}$ and $\frac{1}{(1-x)^2}$ as n grows.

Turn in your definitions and a couple of coefficient printouts to demonstrate that they work.